Universidade de São Paulo
Escola de Artes, Ciências e Humanidades
Sistemas de Informação

Relatório do Exercício-Programa de Assembly

ACH2034 - Organização e Arquitetura de Computadores I

Gustavo Neves Moraes NUSP: 16986195
Vinícius Cunha Pereira NUSP: 16973242

Profa. Gisele S. Craveiro

São Paulo
2026

Sumário

1 Introdução **2**
1.1 A Arquitetura MIPS e seus Princípios de Projeto 2

2 Descrição do problema e código alto nível da solução **4**
2.1 Descrição do problema 4
2.2 Algoritmo de solução 4

3 Implementação em Assembly **6**
3.1 Rotina principal (`main`) 6
3.2 Sub-rotina `mmc` 7
3.3 Sub-rotina `divide` 9

4 Micro-operações do Conjunto de Instruções Utilizado **11**
4.1 Convenções adotadas 11
4.2 Pseudo-instruções e suas expansões 12
4.3 Instrução `ori` 13
4.4 Instrução `lui` 13
4.5 Instrução `syscall` 14
4.6 Instrução `sw` 14
4.7 Instrução `jal` 14
4.8 Instrução `addu` 15
4.9 Instrução `addi` 15
4.10 Instrução `lw` 16
4.11 Instrução `bne` 16
4.12 Instrução `j` 17
4.13 Instrução `beq` 17
4.14 Instrução `mult` 17
4.15 Instrução `mflo` 18
4.16 Instrução `jr` 18
4.17 Instrução `div` 19
4.18 Instrução `mfhi` 19

1 Introdução

O presente trabalho trata da resolução e análise de um problema prático desenvolvido no âmbito da disciplina de Organização e Arquitetura de Computadores. O objetivo do programa consiste no cálculo do Mínimo Múltiplo Comum (MMC) entre dois números inteiros positivos fornecidos pelo usuário, empregando o método de fatoração por tentativa. A solução é estruturada a partir de uma rotina principal (`main`) e das sub-rotinas `mmc` e `divide`, onde a verificação de divisibilidade e a atualização dos ponteiros das variáveis de entrada ocorrem de maneira modular.

A transição de uma descrição em linguagem de alto nível (como o pseudocódigo em C) para a implementação em linguagem de montagem (*Assembly*) MIPS evidencia os aspectos operacionais da arquitetura do computador que permanecem abstratos em linguagens de alto nível. Enquanto uma linguagem como C permite a manipulação direta de ponteiros de memória (`*m` e `*n`) e expressões aritméticas compostas em uma única linha, a arquitetura MIPS exige que cada acesso à memória seja traduzido em instruções explícitas de carga (`lw`) e escrita (`sw`). Além disso, a alocação de variáveis temporárias e o controle do fluxo de execução por meio da pilha de execução (`$sp`) e registradores de retorno (`$ra`) demandam um gerenciamento rigoroso dos recursos do processador.

1.1 A Arquitetura MIPS e seus Princípios de Projeto

Desenvolvida originalmente na Universidade de Stanford em 1981 por John Hennessy e comercializada posteriormente pela MIPS Computer Systems, a arquitetura MIPS (*Microprocessor without Interlocked Pipeline Stages*) é um dos marcos fundamentais do paradigma RISC (*Reduced Instruction Set Computer*). A filosofia central do projeto RISC baseia-se na simplificação do conjunto de instruções para permitir a implementação de *hardware* mais simples, rápido e eficiente em termos de consumo de energia, transferindo a complexidade de otimização para o compilador.

A arquitetura MIPS é orientada por quatro princípios basilares de projeto de *hardware*:

1. **Simplicidade favorece a regularidade:** Todas as instruções MIPS possuem extensão fixa de 32 bits (1 palavra) e formato altamente padronizado, onde instruções aritméticas operam estritamente sobre três operandos. Essa regularidade simplifica a lógica de busca (*fetch*) e decodificação do processador.

2. **Menor é mais rápido:** O MIPS dispõe de um banco de registradores reduzido, composto por 32 registradores de propósito geral de 32 bits. A limitação em 32 unidades reduz a distância física de propagação dos sinais elétricos no silício, garantindo tempos de ciclo de *clock* menores. O uso desses registradores segue a convenção da ABI (*Application Binary Interface*), reservando faixas específicas para argumentos (`$a0-$a3`), valores de retorno (`$v0-$v1`), temporários (`$t0-$t9`), salvos (`$s0-$s7`) e ponteiros de controle (`$sp`, `$ra`).

3. **Otimização do caso comum:** A arquitetura codifica constantes diretamente em instruções imediatas (`addi`, `slti`) e disponibiliza o registrador `$zero` (`$0`) fixado em zero no *hardware*. Isso acelera operações frequentes como inicialização, atribuição e cópia de valores entre registradores sem a necessidade de acessos custosos à memória.

4. **Um bom projeto exige bons compromissos:** Para acomodar operandos imediatos de 16 bits ou endereços de desvio mantendo a extensão fixa de 32 bits para todas as instruções, o MIPS adota três formatos distintos de instrução: Tipo R (Registrador), Tipo I (Imediato) e Tipo J (*Jump*).

Uma característica marcante da arquitetura MIPS é a sua estrutura *Load-Store*. Instruções aritméticas e lógicas operam exclusivamente sobre dados contidos nos registradores internos da CPU; nenhum operando de memória pode ser utilizado diretamente em uma operação de cálculo. Além disso, a memória principal é desempenhada por endereçamento por *byte* com restrição de alinhamento, exigindo que palavras de 32 bits estejam alinhadas em endereços múltiplos de 4. A arquitetura não possui modo de endereçamento indireto por memória, de forma que os endereços efetivos de carga e escrita são sempre calculados no ciclo de execução somando-se um registrador base a um deslocamento imediato (*offset*).

2 Descrição do problema e código alto nível da solução

2.1 Descrição do problema

O problema resolvido neste relatório corresponde à Questão 8 da lista de problemas propostos para o EP.

O item (a) pede a implementação de uma função com assinatura `int divide (int *m, int *n, int d)`, que recebe três inteiros positivos e deve devolver 1 caso o divisor `d` divida pelo menos um dos valores apontados por `m` ou `n`, e 0 caso contrário. Além de retornar esses valores, sempre que `d` divide `*m`, o valor apontado é atualizado para o quociente da divisão, e o mesmo vale para `*n`. Ou seja, se `d` divide `*m`, o valor apontado por `m` passa a ser `*m/d` (além da função retornar 1).

Por sua vez, o item (b) pede um programa que leia dois inteiros positivos `m` e `n` e, utilizando obrigatoriamente a função do item anterior, calcule o Mínimo Múltiplo Comum (MMC) entre eles.

2.2 Algoritmo de solução

A estratégia adotada para resolução do problema foi a de fatoração por tentativa. Testa-se sequencialmente cada divisor candidato `d`, começando em 2. Para cada `d`, chama-se `divide(&m, &n, d)`. Se a chamada indicar que pelo menos um dos dois valores foi dividido, o divisor `d` é incorporado ao acumulador do MMC (multiplicando-o) e o mesmo `d` é testado novamente, pois ele pode dividir novamente algum dos dois valores restantes. Caso `d` não divida nem `*m` nem `*n`, incrementa-se `d` e repete-se o processo. O algoritmo termina quando `m` e `n` foram reduzidos a 1, momento em que todos os seus fatores primos já foram extraídos e incorporados ao acumulador, que corresponde, portanto, ao MMC entre os valores originais.

A seguir, o Código 1 apresenta uma versão em linguagem C do algoritmo, servindo de base para fazer a conexão entre a descrição da resolução acima e a implementação em Assembly apresentada na Seção 3.

```c
int divide(int *m, int *n, int d) {
    int ok = 0;

    if (*m % d == 0) {
        *m /= d;
        ok = 1;
    }
    if (*n % d == 0) {
        *n /= d;
        ok = 1;
    }

    return ok;
}
```

```c

int MMC(int m, int n) {
    int result = 1;
    int d = 2;

    while (m != 1 || n != 1) {
        if (divide(&m, &n, d)) {
            result *= d;
        } else {
            d++;
        }
    }

    return result;
}

int main() {
    int m, n;
    scanf("%i", &m);
    scanf("%i", &n);

    int mmc = MMC(m, n);

    printf("O MMC e: %i\n", mmc);
    return 0;
}
```

Código 1: Pseudocódigo em C do algoritmo

Vale comentar que uma possível otimização para a estratégia adotada (fatoração por tentativa) seria testar apenas divisores primos, em vez de verificar todos os inteiros sequencialmente. Isso porque não faz sentido, testar divisores compostos, como por exemplo 6, já que seus fatores primos (2 e 3) necessariamente teriam sido testados anteriormente. Dessa forma, poderia ser mantido um vetor contendo alguns números primos, e a variável `d` passaria a assumir apenas esses valores, reduzindo a quantidade de testes desnecessários e tornando o processo de fatoração mais eficiente, principalmente para entradas de maior magnitude.

3 Implementação em Assembly

O programa foi desenvolvido em MIPS Assembly e está organizado em três blocos principais. A rotina `main` é responsável por realizar a leitura dos valores de entrada e exibir o resultado final. Além da rotina principal, o programa conta com duas sub-rotinas, isto é, blocos de código que executam tarefas específicas e podem ser chamados por outras partes do programa, retornando o controle ao ponto de chamada após sua execução.

A sub-rotina `mmc` executa o cálculo do MMC por meio de divisões sucessivas. Por sua vez, a sub-rotina `divide` (chamada repetidamente por `mmc`), verifica se o divisor candidato `d` divide os valores `m` e/ou `n`, atualizando esses valores sempre que a divisão é possível.

A Tabela 1 resume o papel de cada registrador utilizado, seguindo a convenção de uso da arquitetura MIPS (registradores `$t` como temporários não preservados entre chamadas, `$s` como registradores salvos/persistentes, `$a` para argumentos e `$v` para valores de retorno).

Registrador	Papel no programa
`$sp`	Ponteiro de pilha, usado para empilhar parâmetros e contexto salvo
`$ra`	Endereço de retorno, salvo/restaurado na pilha em `mmc` e `divide`
`$s0`	Acumulador do MMC (salvo na pilha durante as chamadas a `divide`)
`$s1`	Divisor candidato `d` (salvo na pilha durante as chamadas a `divide`)
`$a0, $a1, $a2`	Parâmetros de `divide` (&m, &n, d) após serem lidos na pilha
`$v0`	Código de *syscall* / valor de retorno de `divide` (0 ou 1)
`$t0, $t1, $t2`	Temporários intermediários (dividendo, resto, quociente)

Tabela 1: Registradores utilizados e seus papéis

3.1 Rotina principal (`main`)

O bloco `main` realiza a leitura dos dois inteiros `m` e `n` via chamadas de sistema. Ele chama a sub-rotina `mmc` e imprime o valor retornado em `$v0`.

```
12 main:
13         # Imprime string
14         li $v0, 4
15         la $a0, msg_m
16         syscall
17
18         # Le inteiro m
19         li $v0, 5
20         syscall
21         sw $v0, m
22
```

```
23          # Imprime string
24          li $v0, 4
25          la $a0, msg_n
26          syscall
27
28          # Ler inteiro n
29          li $v0, 5
30          syscall
31          sw $v0, n
32
33          # Chama a rotina mmc
34          jal mmc
35
36          # Salva o retorno (o MMC calculado) em $t0 temporariamente
37          move $t0, $v0
38
39          # imprime string de msg MMC
40          li $v0, 4
41          la $a0, msg_mmc
42          syscall
43
44          # imprime inteiro do MMC
45          li $v0, 1
46          move $a0, $t0
47          syscall
48
49          # acaba o programa ;D
50          li $v0, 10
51          syscall
```

Código 2: Bloco `main`

3.2 Sub-rotina `mmc`

A sub-rotina `mmc` inicia salvando na pilha o endereço de retorno (`$ra`) e os registradores `$s0` e `$s1`, que precisam sobreviver às chamadas a `divide`. Em seguida executa o laço principal do algoritmo. A cada iteração é verificado se `m` e `n` já foram reduzidos a 1 (rótulo `fim_mmc`), caso contrário, empilha os três parâmetros de `divide` (&m, &n e o divisor candidato `d`, nessa ordem) e realiza a chamada com `jal`. Se pelo menos um dos dois valores foi dividido, o acumulador `$s0` é multiplicado por `d` e o laço reinicia com o mesmo `d`, caso contrário, `d` é incrementado (`nao_dividiu`). Ao final, o resultado é movido para `$v0`, o contexto salvo é restaurado e a sub-rotina retorna com `jr $ra`.

```
56  mmc:
57          # Prologo da funcao mmc: salva o contexto na pilha
58          addi $sp, $sp, -12
59          sw   $ra, 8($sp)  # Salva endereco de retorno
60          sw   $s0, 4($sp)  # Salva $s0
61          sw   $s1, 0($sp)  # Salva $s1
62
```

```
63          li $s0, 1 # $s0 = mmc = 1
64          li $s1, 2 # $s1 = d = 2
65
66  mmc_loop:
67          # carrega m e n
68          lw $t0, m # $t0 = m
69          lw $t1, n # $t1 = n
70
71          # $t2 = minimoMMC = 1
72          li $t2, 1
73
74          # se m!=1 continua tentando dividir
75          bne $t0, $t2, tentaDividir
76
77          # A essa ponto do codigo pelo menos m eh 1
78          # mas talvez n ainda pode ser divisivel
79
80          # se n!=1 continua tentando dividir
81          bne $t1, $t2, tentaDividir
82
83          j fim_mmc
84
85  tentaDividir:
86          # ------------------------------------------------
87          # Params
88          #   offset 8($sp) -> &m   (endereço de m)
89          #   offset 4($sp) -> &n   (endereço de n)
90          #   offset 0($sp) -> d    (valor do divisor testado)
91          # Quem chama abre espaço na pilha, empilha os 3 parâmetros, faz o
92          # "jal" e, ao retornar, desempilha o espaço que abriu.
93          # ------------------------------------------------
94          addi $sp, $sp, -12   # abre espaço para 3 palavras na pilha
95          la   $t3, m
96          sw   $t3, 8($sp)      # empilha &m
97          la   $t3, n
98          sw   $t3, 4($sp)      # empilha &n
99          sw   $s1, 0($sp)      # empilha d
100
101         jal divide
102
103         addi $sp, $sp, 12     # desempilha os parâmetros
104         beqz $v0, nao_dividiu # se não dividiu ninguém, tenta outro d.
105
106     mul $s0, $s0, $s1 # c.c. pelo menos um deles foi dividido.
        Multiplicar o MMC pelo d atual.
107         j mmc_loop
108
109 nao_dividiu:
110         addi $s1, $s1, 1 # d++
111         j mmc_loop
112
113 fim_mmc:
```

```
114         # Prepara retorno da funcao
115         move $v0, $s0      # Coloca o resultado (MMC) em $v0
116
117         # Epilogo: restaura o contexto
118         lw   $s1, 0($sp)
119         lw   $s0, 4($sp)
120         lw   $ra, 8($sp)
121         addi $sp, $sp, 12
122         jr   $ra           # Retorna para main
```

Código 3: Sub-rotina `mmc`

3.3 Sub-rotina `divide`

A sub-rotina `divide` recebe seus três parâmetros pela pilha (em vez de registradores de argumento). Como a própria sub-rotina abre seu próprio espaço de pilha (16 bytes, para salvar `$ra`, `$t0`, `$t1` e `$t2`) antes de acessar os parâmetros empilhados pelo chamador, os deslocamentos usados para lê-los (`24($sp)`, `20($sp)` e `16($sp)`) já contabilizam esse deslocamento adicional.

Após copiar os parâmetros para `$a0`, `$a1` e `$a2`, a lógica de divisão é a mesma de antes, isto é, testa `*m` e, independentemente do resultado, testa `*n`, retornando em `$v0` se pelo menos uma das duas divisões ocorreu.

```
138 divide:
139         addi $sp, $sp, -16
140         sw   $ra, 12($sp)    # salva endereço de retorno
141         sw   $t0, 8($sp)     # salva $t0
142         sw   $t1, 4($sp)     # salva $t1
143         sw   $t2, 0($sp)     # salva $t2
144
145         lw   $a0, 24($sp)    # $a0 = &m
146         lw   $a1, 20($sp)    # $a1 = &n
147         lw   $a2, 16($sp)    # $a2 = d
148
149         li $v0, 0 # retorno eh zero ate que se prove o contrario
150
151         # testa *m
152         lw $t0, 0($a0) # $t0 = *m
153         div $t0, $a2
154         mfhi $t1 # resto
155
156         bnez $t1, testa_n # se resto != 0, não divide
157
158         mflo $t2 # quociente
159         sw $t2, 0($a0) # *m = *m / d
160         li $v0, 1  # retorna 1 pois dividiu
161
162 testa_n:
163         lw $t0, 0($a1) # $t0 = *n
164         div $t0, $a2
```

```
165         mfhi $t1 # resto
166
167         bnez $t1, fim_divide # se resto != 0, termina
168
169         mflo $t2 # quociente
170         sw $t2, 0($a1) # *n = *n / d
171         li $v0, 1 # dividiu
172
173 fim_divide:
174         lw   $t2, 0($sp)
175         lw   $t1, 4($sp)
176         lw   $t0, 8($sp)
177         lw   $ra, 12($sp)
178         addi $sp, $sp, 16
179         jr $ra
```

Código 4: Sub-rotina `divide`

4 Micro-operações do Conjunto de Instruções Utilizado

4.1 Convenções adotadas

Para cada instrução, apresentamos as micro-operações dos três ciclos pedidos (busca, indireto e execução) usando notação de transferência entre registradores (RTL). Vale estabelecer as seguintes convenções usadas, algumas delas são evidenciadas aqui pois não serão repetidas novamente ao decorrer do relatório:

`PC` (***Program Counter***) **-** Registrador que armazena o endereço da próxima instrução a ser executada.

`IR` (***Instruction Register***) **-** Registrador que armazena a instrução atualmente em execução.

`Reg[x]` **-** Conteúdo do registrador de propósito geral `x`.

`MEM[end]` **-** Conteúdo da posição de memória localizada no endereço `end`.

`HI, LO` **-** Registradores especiais utilizados para armazenar, respectivamente, a parte alta e a parte baixa dos resultados das instruções de multiplicação e divisão.

sign_extend **-** Operação que estende um valor imediato de 16 para 32 bits preservando seu sinal.

zero_extend **-** Operação que estende um valor imediato de 16 para 32 bits preenchendo os bits mais significativos com zeros.

Ciclo indireto. A arquitetura MIPS não possui modo de endereçamento indireto, isto é, nenhuma instrução do conjunto usa um endereço armazenado em memória para localizar seu operando. Mesmo as instruções de acesso à memória (`lw`, `sw`) calculam o endereço efetivo diretamente durante a execução, somando um registrador base a um deslocamento imediato. Isso é consistente com a caracterização do MIPS como uma arquitetura *load-store* feita no Apêndice A [**?**], que descreve `c(rx)` (a soma de uma constante ao conteúdo de um registrador base) como o único modo de endereçamento de memória realmente oferecido pela máquina. Por esse motivo, o ciclo indireto não se aplica a absolutamente nenhuma instrução usada neste programa.

Ciclo de execução. A decodificação/leitura de operandos e a escrita do resultado (*write-back*) são apresentadas como parte do ciclo de execução de cada instrução, em vez de fases isoladas.

Repetição de ocorrências. Quando a mesma instrução real aparece mais de uma vez no código (inclusive quando gerada por diferentes pseudo-instruções), a primeira

ocorrência será detalhada por completo. No entanto, as demais são apenas listadas por linha/rótulo. Isso se deve por executarem exatamente a mesma sequência de micro-operações.

4.2 Pseudo-instruções e suas expansões

A Tabela 2 lista as pseudo-instruções adotadas e as instruções reais que o montador (MARS) gera a partir delas. Essa distinção entre instruções reais e pseudo-instruções, assim como a descrição de cada uma das pseudo-instruções usadas, é documentada na Seção A.10 do Apêndice A [**?**], que trata especificamente do assembly do MIPS R2000 simulado pelo SPIM. O detalhamento de micro-operações de cada instrução real gerada está nas subseções correspondentes, a seguir:

Pseudo-instrução	**Exemplo no código**	**Instruções reais geradas**
`li $reg, imed`	`li $v0, 4` (li 14)	`ori $reg, $zero, imed`*
`la $reg, rótulo`	`la $a0, msg_m` (li 15)	`lui $reg, alta(rótulo)`; `ori $reg, $reg, baixa(rótulo)`
`lw $reg, rótulo`	`lw $t0, m` (li 68)	`lui $at, alta(rótulo)`; `lw $reg, baixa(rótulo)($at)`
`sw $reg, rótulo`	`sw $v0, m` (li 21)	`lui $at, alta(rótulo)`; `sw $reg, baixa(rótulo)($at)`
`move $rd, $rs`	`move $t0, $v0` (li 37)	`addu $rd, $rs, $zero`
`beqz $rs, rótulo`	`beqz $v0, nao_dividiu` (li 104)	`beq $rs, $zero, rótulo`
`bnez $rs, rótulo`	`bnez $t1, testa_n` (li 156)	`bne $rs, $zero, rótulo`
`mul $rd, $rs, $rt`	`mul $s0, $s0, $s1` (li 106)	`mult $rs, $rt`; `mflo $rd`

Tabela 2: Pseudo-instruções utilizadas e suas expansões

Nas subseções seguintes, cada instrução real gerada é detalhada individualmente, na ordem de sua primeira aparição no código. Cada descrição segue uma estrutura comum, composta pelo formato da instrução, sua operação em nível de transferência entre registradores (RTL), sua utilização no contexto do programa e um exemplo correspondente à primeira ocorrência no código, seguido das demais ocorrências quando aplicável.

O campo **Formato (I, R ou J)** apresenta a organização dos 32 bits da instrução segundo a arquitetura MIPS, indicando a disposição de seus campos. Entre esses campos, o `opcode` identifica a operação a ser executada pelo processador e, nas instruções do formato R, o campo `funct` complementa essa identificação, distinguindo operações que compartilham o mesmo `opcode`. Os valores apresentados para esses campos seguem a especificação do manual de referência da arquitetura [**?**].

Começamos pelas instruções geradas pela pseudo-instrução `li`, usada para carregar constantes pequenas em registradores.

4.3 Instrução `ori`

A instrução `ori` (*OR Immediate*) realiza uma operação lógica OR entre o conteúdo de um registrador e um valor imediato de 16 bits. Neste programa, ela é utilizada principalmente na expansão da pseudo-instrução `li`, permitindo carregar constantes pequenas diretamente em um registrador.

`ori`

Formato (I): `opcode(6)=001101 | rs(5) | rt(5) | imediato(16)`

Execução: $Reg[rt] \leftarrow Reg[rs] \vee zero_extend(imediato)$

Uso: Implementa a carga de constantes de `li` (com $rs = \$zero$) e a metade menos significativa do endereço em `la` e nas formas de `lw`/`sw` com rótulo direto.

Primeira ocorrência: `li $v0, 4` (linha 14) → `ori $v0, $zero, 4`: $Reg[v0] \leftarrow Reg[zero] \vee zero_extend(4) = 4$.

Demais ocorrências: linhas 15, 19, 24, 25, 29, 40, 41, 45, 50, 63, 64, 68, 69, 72, 95, 97, 149, 160, 171.

4.4 Instrução `lui`

A instrução `lui` (*Load Upper Immediate*) carrega um valor imediato nos 16 bits mais significativos de um registrador, preenchendo os 16 bits inferiores com zeros. No programa, ela é empregada pelo montador na expansão de pseudo-instruções que necessitam construir endereços completos de memória.

`lui`

Formato (I): `opcode(6)=001111 | rs(5, não usado) | rt(5) | imediato(16)`

Execução: $Reg[rt] \leftarrow imediato \ll 16$ (bits menos significativos preenchidos com zero)

Uso: Usada como primeira metade de `la` e das formas de `lw`/`sw` com rótulo direto, para carregar os 16 bits mais significativos do endereço do rótulo.

Primeira ocorrência: primeira metade de `la $a0, msg_m` (linha 15) → `lui $a0, alta(msg_m)`: $Reg[a0] \leftarrow alta(msg_m) \ll 16$.

Demais ocorrências: linhas 21, 25, 31, 41, 68, 69, 95, 97.

4.5 Instrução `syscall`

A instrução `syscall` (*System Call*) é utilizada para solicitar a execução de serviços oferecidos pelo sistema ou, no caso do simulador MARS/SPIM, pelo próprio ambiente de execução. Neste programa, ela é empregada para realizar operações de entrada e saída, como impressão de mensagens, leitura de valores inteiros e encerramento da execução.

`syscall`

Formato (especial): `opcode(6)=000000 | code(20) | funct(6)=001100`

Execução: dispara uma exceção (*trap*) que transfere o controle ao núcleo do simulador. O núcleo lê o código de serviço em `$v0` e executa a rotina correspondente (imprimir string usando o endereço em `$a0`, ler inteiro devolvendo-o em `$v0`, imprimir inteiro usando `$a0`, ou encerrar o programa), retornando o controle à instrução seguinte.

Primeira ocorrência: linha 16, com $Reg[v0] = 4$: imprime a string apontada por `$a0` (mensagem "Digite m: ").

Demais ocorrências: linhas 20, 26, 30, 42, 47, 51.

4.6 Instrução `sw`

A instrução `sw` (*Store Word*) armazena uma palavra de 32 bits de um registrador em uma posição de memória. No programa, ela é utilizada para atualizar as variáveis globais após a execução das operações de cálculo.

`sw`

Formato (I): `opcode(6)=101011 | rs(5) | rt(5) | deslocamento(16)`

Execução: $EA \leftarrow Reg[rs] + sign_extend(deslocamento)$; $MEM[EA] \leftarrow Reg[rt]$

Primeira ocorrência: `sw $v0, m` (linha 21), expandida para `sw $v0, baixa(m)($at)`: $EA \leftarrow Reg[at] + sign_extend(baixa(m))$; $MEM[EA] \leftarrow Reg[v0]$ (guarda o valor de `m` lido do teclado).

Demais ocorrências: linhas 31, 59, 60, 61, 96, 98, 99, 140, 141, 142, 143, 159, 170.

4.7 Instrução `jal`

A instrução `jal` (*Jump And Link*) realiza a chamada de uma sub-rotina, desviando a execução e armazenando o endereço de retorno no registrador `$ra`. No programa, ela é

utilizada para invocar as sub-rotinas responsáveis pelo cálculo do MMC e pelas divisões sucessivas.

`jal`

Formato (J): `opcode(6)=000011 | endereço-alvo(26)`

Execução: $Reg[ra] \leftarrow PC$ (endereço já incrementado na busca, ou seja, o da instrução seguinte); $PC \leftarrow PC[31{:}28] \,\|\, (\text{endereço-alvo} \ll 2)$

Primeira ocorrência: `jal mmc` (linha 34): salva em `$ra` o endereço de retorno para `main` e desvia para o rótulo `mmc`.

Demais ocorrências: linha 101 (`jal divide`).

4.8 Instrução `addu`

A instrução `addu` (*Add Unsigned*) realiza a soma entre dois registradores sem considerar detecção de overflow. Neste programa, ela é utilizada pelo montador para implementar a pseudo-instrução `move`, copiando o conteúdo de um registrador para outro.

`addu`

Formato (R): `opcode(6)=000000 | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6)=100001`

Execução: $Reg[rd] \leftarrow Reg[rs] + Reg[rt]$ (soma sem verificação de *overflow*)

Uso: Implementa `move`, sempre com $rt = \$zero$, de modo que $Reg[rd] \leftarrow Reg[rs]$.

Primeira ocorrência: `move $t0, $v0` (linha 37) → `addu $t0, $v0, $zero`: $Reg[t0] \leftarrow Reg[v0] + 0$ (copia o MMC retornado por `mmc` para um registrador temporário).

Demais ocorrências: linhas 46, 115.

4.9 Instrução `addi`

A instrução `addi` (*Add Immediate*) soma um valor imediato ao conteúdo de um registrador, armazenando o resultado em um registrador de destino. No programa, ela é empregada principalmente para atualizar contadores, ajustar ponteiros de pilha e incrementar variáveis de controle.

`addi`

Formato (I): `opcode(6)=001000 | rs(5) | rt(5) | imediato(16)`

Execução: $Reg[rt] \leftarrow Reg[rs] + sign_extend(imediato)$

Uso: Diferente de `ori`, o imediato aqui é estendido com sinal, e (pela especificação do MIPS) a instrução pode gerar exceção de *overflow* – o que não ocorre neste programa, já que os valores manipulados são pequenos.

Primeira ocorrência: `addi $sp, $sp, -12` (linha 58): $Reg[sp] \leftarrow Reg[sp] + (-12)$, abrindo espaço na pilha para o prólogo de `mmc`.

Demais ocorrências: linhas 94, 103, 110, 121, 139, 178.

4.10 Instrução `lw`

A instrução `lw` (*Load Word*) realiza a leitura de uma palavra de 32 bits da memória para um registrador de propósito geral. Neste programa, ela é utilizada para carregar os valores das variáveis armazenadas em memória antes de sua manipulação pelas sub-rotinas.

`lw`

Formato (I): `opcode(6)=100011 | rs(5) | rt(5) | deslocamento(16)`

Execução: $EA \leftarrow Reg[rs] + sign_extend(deslocamento)$; $Reg[rt] \leftarrow MEM[EA]$

Primeira ocorrência: `lw $t0, m` (linha 68), expandida para `lw $t0, baixa(m)($at)`: $EA \leftarrow Reg[at] + sign_extend(baixa(m))$; $Reg[t0] \leftarrow MEM[EA]$ (carrega o valor atual de `m`).

Demais ocorrências: linhas 69, 118, 119, 120, 145, 146, 147, 152, 163, 174, 175, 176, 177.

4.11 Instrução `bne`

A instrução `bne` (*Branch if Not Equal*) realiza um desvio condicional quando os conteúdos de dois registradores são diferentes. No programa, ela é empregada para controlar laços de repetição e implementar a pseudo-instrução `bnez`.

`bne`

Formato (I): `opcode(6)=000101 | rs(5) | rt(5) | deslocamento(16)`

Execução: se $Reg[rs] \neq Reg[rt]$, então $PC \leftarrow PC + (sign_extend(deslocamento) \ll 2)$; caso contrário PC permanece o valor já incrementado na busca.

Primeira ocorrência: `bne $t0, $t2, tentaDividir` (linha 75): desvia para

`tentaDividir` se `m` (em `$t0`) ainda for diferente de 1.

Demais ocorrências: linha 81 (`bne` direto); linhas 156 e 167 (`bnez` expandido, com $rt = \$zero$).

4.12 Instrução `j`

A instrução `j` (*Jump*) realiza um salto incondicional para um novo endereço de execução. Neste programa, ela é utilizada para alterar o fluxo de controle entre diferentes trechos do algoritmo, sem necessidade de armazenar um endereço de retorno.

`j`

Formato (J): `opcode(6)=000010 | endereço-alvo(26)`

Execução: $PC \leftarrow PC[31{:}28] \,\|\, (\text{endereço-alvo} \ll 2)$

Primeira ocorrência: `j fim_mmc` (linha 83): desvia incondicionalmente para o epílogo de `mmc`.

Demais ocorrências: linhas 107, 111.

4.13 Instrução `beq`

A instrução `beq` (*Branch if Equal*) realiza um desvio condicional quando os conteúdos de dois registradores são iguais. Neste programa, ela é utilizada na implementação de decisões lógicas e na expansão da pseudo-instrução `beqz`.

`beq`

Formato (I): `opcode(6)=000100 | rs(5) | rt(5) | deslocamento(16)`

Execução: se $Reg[rs] = Reg[rt]$, então $PC \leftarrow PC + (sign_extend(deslocamento) \ll 2)$.

Única ocorrência: `beqz $v0, nao_dividiu` (linha 104), expandida para `beq $v0, $zero, nao_dividiu`: desvia se `divide` não dividiu nenhum dos dois valores.

4.14 Instrução `mult`

A instrução `mult` (*Multiply*) realiza a multiplicação de dois operandos armazenados em registradores, gravando o resultado de 64 bits nos registradores especiais `HI` e `LO`. No programa, ela faz parte da expansão da pseudo-instrução `mul`.

mult

Formato (R especial): `opcode(6)=000000 | rs(5) | rt(5) | (não usado, 10) | funct(6)=011000` – não grava em registrador de propósito geral.

Execução: $\{HI, LO\} \leftarrow Reg[rs] \times Reg[rt]$ (produto de 64 bits com sinal; `HI` recebe a parte alta, `LO` a parte baixa)

Única ocorrência: primeira metade de `mul $s0, $s0, $s1` (linha 106) → `mult $s0, $s1`: $\{HI, LO\} \leftarrow Reg[s0] \times Reg[s1]$.

4.15 Instrução `mflo`

A instrução `mflo` (*Move From LO*) copia para um registrador de propósito geral o conteúdo do registrador especial `LO`. Neste programa, ela é utilizada para recuperar o resultado das multiplicações realizadas pela instrução `mult`.

mflo

Formato (R especial): `opcode(6)=000000 | (não usado, 10) | rd(5) | (não usado, 5) | funct(6)=010010`

Execução: $Reg[rd] \leftarrow LO$

Primeira ocorrência: segunda metade de `mul $s0, $s0, $s1` (linha 106) → `mflo $s0`: $Reg[s0] \leftarrow LO$ (acumula o divisor no MMC).

Demais ocorrências: linhas 158, 169 (quociente da divisão em `divide`).

4.16 Instrução `jr`

A instrução `jr` (*Jump Register*) transfere o controle para o endereço contido em um registrador. Neste programa, ela é utilizada para retornar de sub-rotinas por meio do endereço previamente armazenado em `$ra`.

jr

Formato (R): `opcode(6)=000000 | rs(5) | (não usado, 15) | funct(6)=001000`

Execução: $PC \leftarrow Reg[rs]$

Primeira ocorrência: `jr $ra` (linha 122): retorna de `mmc` para `main`, usando o endereço restaurado da pilha.

Demais ocorrências: linha 179 (retorno de `divide`).

4.17 Instrução `div`

A instrução `div` (*Divide*) realiza a divisão inteira entre dois registradores, armazenando o quociente em `LO` e o resto em `HI`. No programa, ela é utilizada para verificar se um divisor candidato divide exatamente os valores utilizados no cálculo do MMC.

`div`

Formato (R especial): `opcode(6)=000000 | rs(5) | rt(5) | (não usado, 10) | funct(6)=011010`

Execução: $LO \leftarrow Reg[rs] \div Reg[rt]$ (quociente, com sinal); $HI \leftarrow Reg[rs] \bmod Reg[rt]$ (resto)

Primeira ocorrência: `div $t0, $a2` (linha 153): $LO \leftarrow Reg[t0] \div Reg[a2]$; $HI \leftarrow Reg[t0] \bmod Reg[a2]$ (testa se `*m` é divisível por `d`).

Demais ocorrências: linha 164 (mesmo teste para `*n`).

4.18 Instrução `mfhi`

A instrução `mfhi` (*Move From HI*) copia para um registrador de propósito geral o conteúdo do registrador especial `HI`. Neste programa, ela é utilizada para verificar o resto produzido pela instrução `div`, permitindo determinar se uma divisão foi exata.

`mfhi`

Formato (R especial): `opcode(6)=000000 | (não usado, 10) | rd(5) | (não usado, 5) | funct(6)=010000`

Execução: $Reg[rd] \leftarrow HI$

Primeira ocorrência: `mfhi $t1` (linha 154): $Reg[t1] \leftarrow HI$ (recupera o resto da divisão de `*m` por `d`).

Demais ocorrências: linha 165 (resto da divisão de `*n`).